

# ASSIGNMENT No 1 — Operating Systems (CE80363)

---

**Student:** \_\_\_\_

**Date:** \_\_\_\_

**Lecturer:** Placide RWEGO MUCYO

---

## Question 1: Discuss on Computer System Architecture

---

### 1.1 Meaning

Computer system architecture refers to the abstract design and the structural organisation of a computer system — that is, how the fundamental hardware components (processor, memory, input/output devices and buses) are organised and how they work together to execute programs. It defines the internal functional relationship between the hardware and the system software, and it determines how instructions and data move inside the machine.

Almost all modern computers follow the **stored-program (Von Neumann) architecture**, in which both the program instructions and the data are stored in the same main memory. The computer does not need to be rewired for each task; it only loads a different program into memory.

### 1.2 Major components of a computer system

A computer system is made of two main parts: **hardware** and **software**.

#### A. Hardware

1. **Central Processing Unit (CPU)** — the "brain" of the computer; it executes instructions. It contains:
2. **Arithmetic and Logic Unit (ALU):** performs arithmetic (+, −, ×, ÷) and logical (AND, OR, comparison) operations.
3. **Control Unit (CU):** fetches instructions from memory, decodes them and controls/coordinates the other components of the system.
4. **Registers:** small, very fast storage locations inside the CPU (e.g. the Program Counter which holds the address of the next instruction, the Accumulator, and general-purpose registers).

5. **Main memory (RAM/ROM)** — holds the instructions and data currently being used. RAM is volatile (contents are lost when power is off), ROM is non-volatile and holds the bootstrap/loader program.
6. **Input devices** — keyboard, mouse, scanner, sensor: they feed data and commands into the system.
7. **Output devices** — monitor, printer, speaker: they present processed results to the user.
8. **Secondary/storage devices** — hard disk, SSD, USB flash drive, CD/DVD: they keep programs and data permanently for later use.
9. **System bus** — a set of electrical paths that interconnect the components:
10. **Data bus** — carries actual data.
11. **Address bus** — carries the memory address being accessed.
12. **Control bus** — carries control signals (read/write, interrupt, clock).

## **B. Software**

- **System software** (Operating System, translators, utility programs) — controls and manages the hardware.
- **Application software** (word processor, browser, games) — performs the user's specific tasks.

### **1.3 How the components work together (instruction cycle)**

The CPU continuously repeats the **fetch-decode-execute cycle**:

1. **Fetch:** the Control Unit takes the next instruction from memory (address held in the Program Counter).
2. **Decode:** it interprets/decodes the operation code of that instruction.
3. **Execute:** the ALU/CU performs the required operation; the result is stored in a register or in memory.
4. The Program Counter is updated and the cycle repeats.

Data is transferred between the CPU, memory and I/O devices through the system bus. Since the CPU works much faster than memory and I/O devices, the operating system and supporting hardware (buffers, caches, channels, interrupts, DMA) are used to keep the CPU busy and to coordinate the different speeds of the components.

### **1.4 Memory hierarchy and the position of the operating system**

Because memory is small and volatile while secondary storage is large and slow, a **memory hierarchy** is used: registers → cache → main memory →

secondary storage, the faster (and more expensive) memories being the smallest.

The machine is organised in layers:

User → Application Programs → Operating System → Hardware

The **operating system (OS)** is system software that sits directly on top of the hardware and provides services to the applications and the user. The user never touches the hardware directly: for example, when a user saves a document, the application requests the OS, and the OS communicates with the appropriate hardware (disk) to write the data. Without the OS, every program would have to manage the CPU, memory, disks and devices itself, which would be inefficient and insecure.

Other architectural ideas related to the OS are:

- **Multiprogramming** — several jobs are kept in memory so that the CPU always has something to execute (keeps CPU and I/O devices busy).
- **Timesharing/multitasking** — the CPU switches between jobs so quickly that each user/process feels it is running alone (response time < 1 second).
- **32-bit vs 64-bit architecture** — determines how much memory the processor can address (a 64-bit architecture can address far more memory and process more data at once).

## 1.5 Conclusion

In summary, computer system architecture describes how the processor, memory, I/O devices and buses are organised to execute programs through the fetch-decode-execute cycle. The operating system is built on this hardware architecture and acts as the interface that hides hardware complexity, allocates resources and provides a convenient, reliable and secure environment for users and application programs.

---

## Question 2: Explain the Operating System Structure

---

### 2.1 Meaning

**Operating System Structure** refers to the way the components of an operating system are organised and how they communicate with one another. A good OS structure makes the system **efficient, reliable, secure, maintainable and easier to develop**. The choice of structure involves

trade-offs among performance, security, reliability, flexibility and maintainability.

The major operating system structures are: simple structure, monolithic structure, layered structure, microkernel structure, modular structure, hybrid structure and virtual-machine structure.

## 2.2 Types of operating system structures

### 1. Simple Structure

- There is little separation between the different components of the system.
- Operating-system functions may be combined into a relatively small and loosely organised program.
- Components may directly interact with the hardware.
- Easy to implement, but difficult to maintain and to secure.
- **Example:** early operating systems such as MS-DOS.

### 2. Monolithic Structure

- A monolithic OS places most operating-system services inside a **single large kernel**.
- The kernel contains: process management, memory management, file-system management, device drivers, networking and security mechanisms.
- All these services operate within the kernel's privileged address space, so they run very fast; however, a fault in one part can crash the whole system.
- **Examples:** traditional UNIX; Linux (monolithic but also supports modular components).

### 3. Layered Structure

- The OS is divided into several **layers (levels)**.
- Each layer provides services to the layer above it and uses the services of the layer below it.
- Advantages: easier to design, implement and debug, because each layer can be tested separately and modification of one layer does not affect the others.
- Disadvantage: extra overhead is caused by the communication between layers.

#### 4. Microkernel Structure

- A microkernel keeps **only the most essential functions** inside the kernel: basic process management, inter-process communication (IPC), basic memory management and low-level hardware management.
- Other services (file systems, device drivers, networking) run **outside the kernel**, in user space, as separate processes/servers.
- Advantages: reliability, security and portability (a fault in a server does not crash the kernel); easy to extend.
- Disadvantage: communication between user-space services and the kernel is slower (message passing), so performance may be lower than a monolithic kernel.

#### 5. Modular Structure

- The OS uses a **core kernel plus separate modules** that can be loaded or removed when required (loadable kernel modules).
- A module may provide a device driver, a file system, network functionality or hardware support.
- Advantages: modularity, flexibility — new functionality is added without recompiling the whole kernel.
- **Example:** Linux supports loadable kernel modules.

#### 6. Hybrid Structure

- A hybrid OS **combines characteristics of different structures**, commonly combining monolithic and microkernel concepts.
- It tries to achieve high performance, modularity, reliability and flexibility at the same time.
- **Examples:** Windows uses a hybrid kernel architecture; Apple's XNU kernel combines Mach-based components with BSD and other components.

#### 7. Virtual-Machine Structure

- A virtual machine (VM) creates an environment that behaves like an independent computer.
- A **hypervisor** manages the underlying physical hardware and allows several virtual machines to run on the same physical computer, each with its own virtual OS.
- Advantages: full isolation, easy testing and better use of the hardware.

## 2.3 Structural support for multiprogramming and timesharing

The structure of the OS also has to support:

- **Multiprogramming** (needed for efficiency): a single user cannot keep the CPU and I/O devices busy at all times, so the OS organises jobs (code and data) so that the CPU always has one job to execute. A subset of the total jobs is kept in memory; when a selected job must wait for I/O, the OS switches to another job (job scheduling).
- **Timesharing (multitasking)**: a logical extension in which the CPU switches jobs so frequently that users can interact with each job while it is running (interactive computing; response time < 1 second). Each user has at least one process in memory, CPU scheduling is performed when several processes are ready, swapping moves processes in and out of memory, and virtual memory allows execution of processes that are not completely in memory.

## 2.4 Comparison / conclusion

| Structure       | Main idea                                             | Example                        |
|-----------------|-------------------------------------------------------|--------------------------------|
| Simple          | Little separation between components                  | MS-DOS                         |
| Monolithic      | All services in one large kernel                      | UNIX, Linux                    |
| Layered         | OS split into levels/layers                           | UNIX System V (layered design) |
| Microkernel     | Only essential services in kernel, rest in user space | MINIX, Mach                    |
| Modular         | Core kernel + loadable modules                        | Linux                          |
| Hybrid          | Combination of monolithic and microkernel             | Windows, macOS (XNU)           |
| Virtual machine | Hypervisor runs several virtual computers             | VMware, VirtualBox, KVM        |

In conclusion, the operating system structure determines how OS components are organised, isolated and communicate; no single structure is perfect, so modern operating systems choose the structure (or combination of structures) that gives the best balance of performance, security, reliability, flexibility and maintainability for their intended use.

## Question 3: Discuss on Major Functions of Operating System

---

The operating system performs many functions. The major ones are discussed below.

### 1. Process Management

- A **process** is a program that is currently being executed (a program is passive, a process is active).
- The OS is responsible for **creating, scheduling, executing, synchronising and terminating processes**, and it ensures that processes receive the necessary resources while maintaining efficient and safe execution.
- Its responsibilities include: creating and deleting processes; scheduling processes; suspending and resuming processes; managing process states (new, ready, running, waiting, terminated); performing **context switching** (saving the state of the running process in its PCB, loading the state of the next process); providing **inter-process communication (IPC)** (shared memory and message passing); providing **process synchronisation** (critical section, race condition, mutex, semaphore, monitor); allocating resources; handling termination; and protecting processes from unauthorised access.
- The OS keeps information about every process in a **Process Control Block (PCB)**: PID, process state, program counter, CPU registers, scheduling information, memory-management information, accounting information and I/O status.
- **Schedulers**: long-term (job) scheduler loads processes from secondary storage into memory and controls the degree of multiprogramming; medium-term scheduler swaps processes out/in of memory; short-term (CPU) scheduler selects a ready process and assigns the CPU to it (using algorithms such as FCFS, SJF, SRTF, Priority, Round Robin, Multilevel Queue and Multilevel Feedback Queue).
- **Threads**: a thread is the smallest unit of CPU execution inside a process; threads give better responsiveness, resource sharing, lower overhead and better use of multicore processors.

### 2. Memory Management

- Memory management keeps track of every byte of main memory, decides which process occupies which part of memory, and reclaims (deallocates) memory when it is no longer needed.

- Functions include: allocation and de-allocation of memory blocks to processes, **logical-to-physical address translation**, protection of the memory of one process against access by another, **virtual memory** (allowing a process to run even if it is not completely in memory) and **swapping** (moving processes in and out of main memory).
- It improves CPU utilisation and degree of multiprogramming and allows efficient and controlled sharing of memory.

### 3. File-System Management

- The file system stores and organises programs and data on secondary storage.
- The OS provides services to **create, delete, open, close, read, write and rename** files and directories; maintains **directory structure** so that files can be found easily; manages **file attributes** (name, type, size, location, protection) and **file organisation** (sequential, indexed, hashed).
- It also handles **file protection and access control** (who may read, write or execute a file) and provides a uniform logical view of storage to the user.

### 4. Device (I/O) Management

- The OS controls input/output devices through **device drivers** and performs **I/O management** so that devices are used efficiently and other processes are not disturbed.
- Functions include: allocating and de-allocating devices, keeping **I/O queues**, providing **device independence** (programs use devices without knowing their physical details), **buffering, caching and spooling** (e.g. spooling print jobs), and handling interrupts from devices.

### 5. Security and Protection

- The OS protects system resources and user data by enforcing **access control** and **authentication** (user ID and password, biometrics).
- It provides permissions/privileges, protects against unauthorised access, viruses and other threats, and isolates users and processes from one another.
- This guarantees confidentiality, integrity and availability of the data.

## 6. User Interface

- The OS provides the interface through which the user interacts with the computer:
- **GUI (Graphical User Interface)** — windows, icons, menus, pointer (WIMP), e.g. Windows and macOS desktops.
- **CLI (Command Line Interface)** — the user types text commands, e.g. the Linux/UNIX shell.
- It makes the computer system convenient to use and hides hardware complexity.

## 7. Communication and Networking

- The OS supports communication between programs and between computers through **IPC mechanisms** (shared memory, message passing, pipes, sockets) and through **networking services** (sending/receiving e-mail, browsing, file transfer, remote login, distributed processing).

## 8. Other functions

- **Error detection and handling:** the OS detects hardware errors (faulty memory, I/O device failure) and software errors and takes corrective action so that the system keeps running.
- **Accounting/accounting information:** records the resources used by each user and process (CPU time, memory, files) for billing and statistics.
- **Job/task management:** keeps track of the status of jobs and schedules them.
- **Command interpretation (shell):** interprets the commands given by the user and starts the requested programs.

## Conclusion

In summary, an operating system is the foundation of a computer system. It manages hardware resources (processes, memory, files, devices), provides services to applications, controls program execution, provides security and user interfaces, and improves system efficiency and reliability. Understanding these functions is essential for studying computer architecture, networking, cybersecurity, cloud computing and software development.

---

*Prepared from: Topic 1 (Introduction to Operating Systems), Topic 2 (Operating System Structures) and Topic 3 (Process Management) — CE80363.*